

Lab 13 Pair Programming – Abstraction, Modularity and Dependency

Aim

This week pair programming lab is focused on applying Object-Oriented Programming principles, namely Abstraction, Modularity and Dependency to solve problems.

在计算机实验室学习编程时，培养计算思维技能的一种有效方法是采用“结对编程”。在这种方法中，两名学生合作解决同一个问题。一名学生扮演“驾驶员”的角色，负责编写代码并将设计概念转化为实际的代码实现。与此同时，另一名学生，被称为“观察员”，他审查代码，识别错误，并提供设计、改进建议。在整个会话中，“驾驶员”和“观察员”的角色不断交替。

Pair Programming Lab:

1. The Lab TA will create a pair within the group.
2. Talk to your pair, who is going to be the Driver first.
3. Complete Task 1 with **Pair Programming** (30-45 minutes).
4. Show the TA the results of your works in Task 1.
5. **Switch roles:** the Observer becomes the Driver and vice versa.
6. Complete Task 2 with **Pair Programming** (30-45 minutes).
7. Show the TA the results of your works in Task 2.
8. TA gives grade.

Pair Programming Lab 13 Task 1 – Abstraction

- **Abstraction** in object-oriented programming is a fundamental concept that involves *simplifying* complex systems by modeling classes based on essential features/functionalities and hiding unnecessary details.
It allows programmers to focus on relevant aspects of an object while ignoring irrelevant details.
- Write a Java program that consist of **more than 3 classes** to model a **Banking System**.
- Plan and write on a piece of paper first: what the *classes* are, what the *instance variables* are.
- What are the *essential features / functionalities* that you choose to implement?
- What are *unnecessary details* that you choose to hide / ignore?

Pair Programming Lab 13 Task 2 – Modularity and Dependency

- Switch role Driver $\leftrightarrow$ Observer.
- **Modularity** is the concept of breaking down a software system into smaller, independent, and interchangeable modules or components.
Each module encapsulates a specific functionality or set of related functionalities, making the system easier to understand, maintain, and scale.

- **Dependency** refers to the relationship between modules when one module relies on the functionality provided by another module. Dependencies can be either direct or indirect, and understanding and managing them is crucial for designing robust and maintainable software.
- Write a Java program that consist of **more than 3 classes** to model an **Online Shopping System**.
- Plan and write on a piece of paper first: what the *classes* are, what the *instance variables* are, such as a **ShoppingCart** that stores a List of **Products**
- What are the classes/modules?
What do they represent?
What are their specific functionalities / a set of related functionalities?
- Have a main class called a **ShoppingSystem** that simulates some online transactions – then explain how the classes/modules depend on each other!

This is the end of CPT111-2324 Pair Programming Lab 13 Task Sheet.